Sixty seconds of silence is not a crash

Animated explainer · src/heartbeat.js

A model request can legitimately take a minute. On a projector, a minute of silence is indistinguishable from a dead program. heartbeat.js prints one line while the request is in flight, then stops.

6 scenes · 5s default interval · one timer · stops on both paths

CONTENTS

What is in this explainer

01 · Why this file exists 3

02 · The timer 4

03 · The lifecycle 5

04 · The unref detail 6

05 · The output 7

06 · Recap 8

Page numbers refer to the folio at the foot of each page.

01 · Why this file exists

Silence looks like a crash.

SRC/HEARTBEAT.JS · *source*

```
export function heartbeat(log, everyMs = 5000) {
  const started = Date.now();
  const timer = setInterval(() => {
    const seconds = Math.round((Date.now() - started) / 1000);
    log(ui.waiting('  … still waiting for the model (' + seconds + 's)'));
  }, everyMs);
  timer.unref?.();
  return { stop: () => clearInterval(timer) };
}
```

Arm it — Before the request starts.

Wait — One line every five seconds.

Stop it — On success and on failure.

> **remember** Liveness is a separate concern with its own lifecycle. That is why it is not inside the loop.
> **On screen:** A request can take 60 seconds. Silence on stage looks like a crash. heartbeat starts a timer that prints one readable line every few seconds. It lives outside the loop so runTask still reads as the loop, not the loop plus a stopwatch.

02 · The timer

One timer. One honest line.

SRC/HEARTBEAT.JS · *source*

```
const started = Date.now();

const timer = setInterval(() => {
  const seconds = Math.round((Date.now() - started) / 1000);
  log(ui.waiting('  … still waiting for the model (' + seconds + 's)'));
}, everyMs);

return { stop: () => clearInterval(timer) };
```

WHAT THE ROOM SEES · *text*

```
── step 1/12 ──────────────────────────
  … still waiting for the model (5s)
  … still waiting for the model (10s)
think: Looking first: what is in the workspace...
```

> **On screen:** The timer captures a start time and prints the elapsed seconds. It returns a tiny handle with one method: stop. The line is deliberately quiet. It says the program is alive without pretending the model is fast.

03 · The lifecycle

Arm it. Stop it on every path.

1. **Announce** — step
2. **heartbeat()** — arm
3. **chatFn** — await
4. **beat.stop()** — finally path

SRC/AGENT.JS · *source*

```
const beat = heartbeat(log, heartbeatMs);
try {
  reply = await chatFn({ messages, tools: TOOL_SCHEMAS });
} catch (err) {
  beat.stop();
  // ...request_failed...
  return { ok: false, reason: 'request_failed', messages };
}
beat.stop();
```

> The stop call is repeated on the catch path. A timer that outlives the request would print into the next scene.
> **On screen:** The loop arms the heartbeat immediately before the model call. On success it stops the timer. On failure it stops the timer first, then returns. Both paths matter. A leaked timer would keep narrating after the run has moved on.

04 · The unref detail

Do not hold the event loop open.

SRC/HEARTBEAT.JS · *source*

```
// A timer that holds the event loop open would turn a clean exit into
// a program that hangs for another five seconds.
timer.unref?.();
```

- 🪧 Normal timer keeps Node alive until it is cleared.
- 🎀 unref() tells Node this timer does not need to keep the process alive.
- 🏁 Clean exit the process can end immediately when the work is done.

> Optional chaining keeps the helper usable in environments where the timer handle does not implement unref.
> **On screen:** This one line is easy to miss and important: unref. A normal interval keeps the Node process alive. A clean exit should not wait for it. unref makes the timer invisible to the event-loop count. The program exits when the work is done.

05 · The output

A quiet sign of life.

A SLOW REQUEST · *text*

```
── step 3/12 ──────────────────────────
  … still waiting for the model (5s)
  … still waiting for the model (10s)
  … still waiting for the model (15s)
think: Now run it - a real shell, a real exit code.
```

> **the point** The heartbeat is not progress. It is proof that the program has not frozen.
> **On screen:** On a slow request the console gains one line every five seconds. When the reply arrives, the lines stop and the normal narration continues. The heartbeat does not claim progress. It only proves the process is alive.

06 · Recap

A tiny file with a clean lifecycle.

• 5s default interval

• 1 timer

• 2 paths stopped

→

Arm before the wait. Start the timer immediately before the request.

→

Stop on success and failure. Both paths clear the interval.

→

unref the handle. Do not keep a finished program alive.

→

Keep it out of the loop. Liveness is its own concern.

> **remember** A slow model is not a crash. Say so, quietly, once every few seconds.
> **On screen:** Five seconds, one timer, two paths stopped. Arm it, wait, stop it on success and failure, and unref the handle. That tiny lifecycle makes a slow model feel like a working program.